Studi Kasus C7

Desain dan Analisis Algoritma

Teknik Informatika Kelas C

Nama:

Muhammad Fakhri Aldiansyah (245150201111049)

Dosen:

Mahendra Data, S.Kom., M.Kom., Ph.D.

Program Studi Teknik Informatika
Jurusan Teknik Informatika
Universitas Brawijaya

2026

Pengantar Studi Kasus:

Berdasarkan studi kasus Travelling Salesman Problem (TSP) menggunakan Algoritma Brute Force, tujuannya adalah mencari permutasi π yang meminimalkan Jarak Total dengan rumus yaitu:

$$\text{Jarak Total} = \sum_{i=1}^{n-1} d_{\pi(i),\pi(i+1)} + d_{\pi(n),\pi(1)}$$

Implementasi Algoritma Brute Force ini berperan dalam melakukan optimasi kecil namun perlu memperhatikan terkait mengunci kota awal. Maksudnya adalah jika memiliki n kota, maka tidak perlu mencari n! permutasi. Karena rutenya berbentuk siklus harus kembali ke awal memulai dari kota 0, 1, 2, atau 3 akan menghasilkan jarak siklus yang sama. Sehingga perlu menetapkan kota 0 sebagai titik awal permanen kemudian hanya perlu mencari permutasi dari sisa kotanya yaitu sebanyak (n-1)! kemungkinan.

1. Pseudocode pada Fungsi :

Pseudocode

```
Fungsi TSP_BruteForce(matriks_jarak D):
    n = panjang baris matriks D
    minimum_jarak = TAK_TERHINGGA
    rute_terbaik = array kosong
    kota_tersisa = [1, 2, ..., n-1]

    // Fungsi menghasilkan semua permutasi dan menghitung jaraknya
secara langsung
    Cari_Permutasi(kota_tersisa, 0, n-1, D, minimum_jarak,
rute_terbaik)
    Kembalikan rute_terbaik dan minimum_jarak


Fungsi Cari_Permutasi(array A, int l, int r, matriks_jarak D,
minimum_jarak, rute_terbaik):
    // Basis rekursi (jika sudah terbentuk 1 permutasi utuh)
    Jika l == r maka:
        jarak_saat_ini = 0
        kota_sebelumnya = 0

        // Menghitung jarak dari kota 0 melintasi permutasi A
        Untuk setiap kota K dalam array A:
            jarak_saat_ini = jarak_saat_ini + D[kota_sebelumnya][K]
            kota_sebelumnya = K

        jarak_saat_ini = jarak_saat_ini + D[kota_sebelumnya][0]

        Jika jarak_saat_ini < minimum_jarak maka:
            minimum_jarak = jarak_saat_ini
            rute_terbaik = salin(A)

    // Langkah rekursif (menukar elemen untuk membuat permutasi baru)
    Selain itu:
```

```
        Untuk i dari l hingga r:
            Tukar elemen A[l] dengan A[i]
            Cari_Permutasi(A, l+1, r, D, minimum_jarak, rute_terbaik)
            Tukar kembali elemen A[l] dengan A[i]
```

Pseudocode yang telah saya buat ini berfokus pada pendekatan Exhaustive Search yang merupakan karakteristik utama dari algoritma Brute Force. Fungsi memanggil dirinya sendiri secara rekursif untuk membangkitkan semua kemungkinan permutasi kota yang akan dikunjungi. Di dalam perulangan terdapat proses menukar elemen (swap) dan menukar kembali elemen tersebut ke posisi semula setelah pemanggilan fungsi rekursif selesai. Hal ini dilakukan agar bisa mengevaluasi kombinasi rute lain tanpa harus membuat array baru di dalam memori, sehingga penggunaan ruang memori tetap efisien meskipun algoritma ini memeriksa seluruh kemungkinan solusi secara langsung untuk menjamin kebenaran hasilnya.

2. Implementasi Program:

TSPBruteForce.java

```java
import java.util.Arrays;

public class TSPBruteForce {
    static int minimumJarak = Integer.MAX_VALUE;
    static int[] ruteTerbaik;
    public static void main(String[] args) {
        int[][] distances = {
            {0, 10, 15, 20},
            {10, 0, 35, 25},
            {15, 35, 0, 30},
            {20, 25, 30, 0}
        };

        int n = distances.length;
        int[] kotaTersisa = new int[n - 1];
        for(int i = 0; i < n - 1; i++) {
            kotaTersisa[i] = i + 1;
        }

        ruteTerbaik = new int[n - 1];
        cariPermutasi(kotaTersisa, 0, n - 2, distances);

        System.out.println("=== Hasil TSP dengan Brute Force ===");
        System.out.print("Rute Terbaik  : 0 -> ");
        for(int kota : ruteTerbaik) {
            System.out.print(kota + " -> ");
        }
        System.out.println("0");
        System.out.println("Minimum Jarak : " + minimumJarak);
    }

    static void cariPermutasi(int[] A, int l, int r, int[][] D) {
```

```
        if(l == r) {
            int jarakSaatIni = 0;
            int kotaSebelumnya = 0;
            for(int k : A) {
                jarakSaatIni += D[kotaSebelumnya][k];
                kotaSebelumnya = k;
            }

            jarakSaatIni += D[kotaSebelumnya][0];

            if(jarakSaatIni < minimumJarak) {
                minimumJarak = jarakSaatIni;
                ruteTerbaik = A.clone();
            }
        }
        else {
            for(int i = l; i <= r; i++) {
                tukar(A, l, i);
                cariPermutasi(A, l + 1, r, D);
                tukar(A, l, i);
            }
        }
    }

    static void tukar(int[] A, int i, int j) {
        int temp = A[i];
        A[i] = A[j];
        A[j] = temp;
    }
}

// Muhammad Fakhri Aldiansyah
// 245150201111049
// DAA TIF C
```

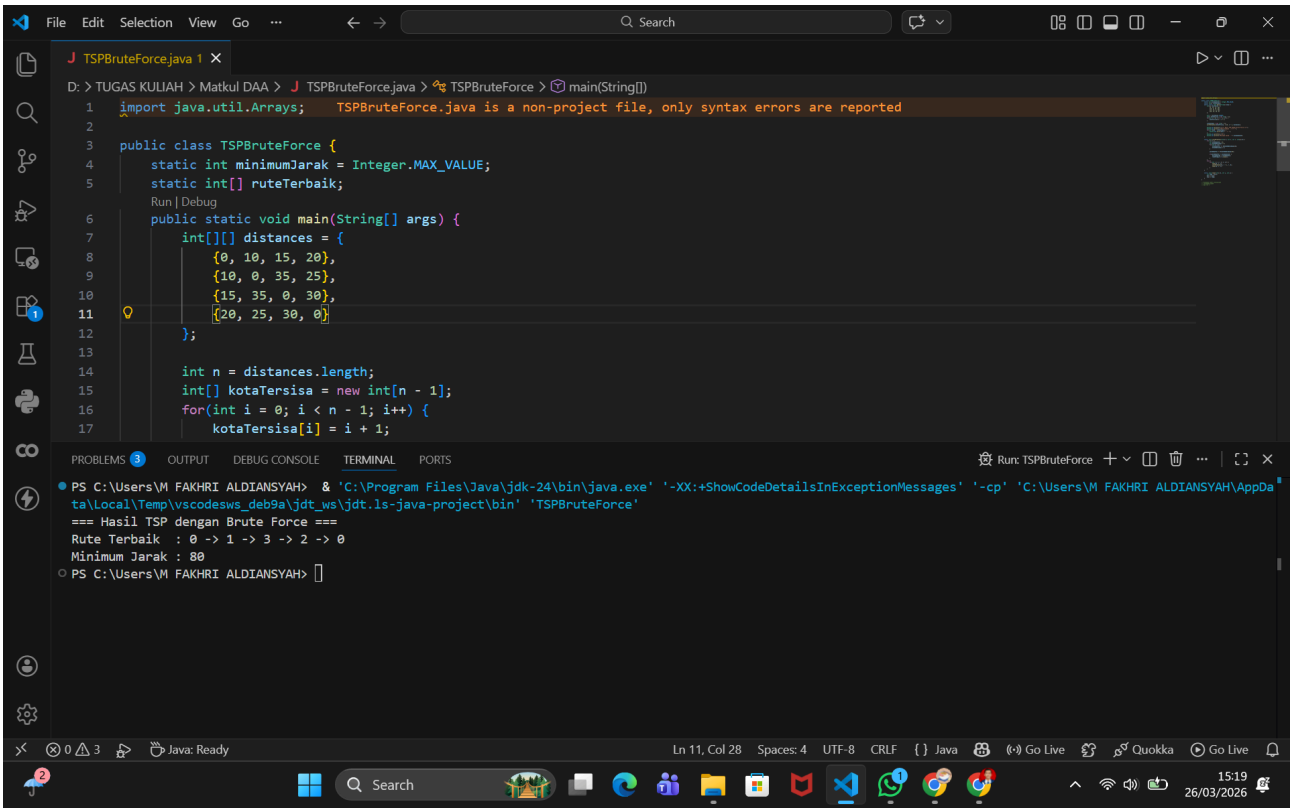
Algoritma Brute Force untuk menyelesaikan Travelling Salesman Problem (TSP) pada studi kasus ini diimplementasikan menggunakan struktur program yang dibangun mengacu pada pseudocode yang telah dirancang sebelumnya dengan memisahkan fungsi utama untuk inisialisasi matriks jarak antar kota dan fungsi rekursif untuk membangkitkan seluruh permutasi rute perjalanan secara Exhaustive Search.

Implementasi program dalam bentuk Java yang telah saya buat ini intinya menerapkan method distances sebagai representasi matriks D yang mana distances[i] [j] adalah jarak dari kota i ke kota j. Kemudian pada fungsi main saya membuat array kotaTersisa berisi {1,2,3} yang mengimplementasikan konsep optimasi yaitu menetapkan kota 0 sebagai titik awal supaya tidak perlu mencari permutasi yang berulang jarak siklusnya. Method A.clone() ini ketika menemukan jarak yang lebih kecil (jarakSaatIni < minimumJarak) ini harus menyalin isi array A ke variabel ruteTerbaik. Jika tidak menggunakan .clone() maka program hanya menyalin alamat memorinya sehingga saat A berubah lagi di rekursi berikutnya maka ruteTerbaik ikut berantakan.

Source code lengkap dari implementasi program Java saya ini telah dikelola menggunakan version control dan dapat diakses secara publik melalui tautan repositori berikut:

Link Repository: https://github.com/punyafakhri/DAA-Studi-Kasus-C7

Program dieksekusi menggunakan input array 2D yang merepresentasikan jarak antar 4 kota sesuai dengan parameter studi kasus. Berikut adalah tangkapan layar dari hasil output eksekusi program yang terlampir pada Gambar 1.



Gambar 1. Hasil Implementasi Program TSPBruteForce

Berdasarkan hasil eksekusi program di atas, Algoritma Brute Force berhasil mengevaluasi seluruh kemungkinan permutasi rute dan menemukan bahwa rute perjalanan paling optimal adalah dimulai dari kota 0 -> 1 -> 3 -> 2 dan kembali lagi ke kota 0. Total jarak minimum yang dihasilkan untuk rute tersebut adalah 80.

3. Analisis Kompleksitas Algoritma

a. Analisis Kompleksitas Waktu (Time Complexity Analysis)

Analisis kompleksitas waktu dilakukan dengan menghitung jumlah operasi dasar sebagai fungsi dari ukuran input n jumlah kota. Pada algoritma Travelling Salesman Problem (TSP) ini perlu menetapkan kota awal secara permanen (kota 0), sehingga algoritma hanya melakukan permutasi terhadap n - 1 kota yang tersisa. Misalkan k = n - 1, fungsi rekursif cariPermutasi membangkitkan seluruh kemungkinan permutasi. Model matematis untuk total waktu eksekusi T(k) dapat didefinisikan sebagai hasil kali antara jumlah permutasi dengan biaya operasi untuk mengevaluasi satu permutasi.

1. Persamaan Rekurensi:

Pada blok else, fungsi melakukan perulangan sebanyak elemen yang tersisa (k) untuk menukar posisi elemen (swap) dan memanggil dirinya sendiri dengan ukuran masalah yang berkurang satu (k - 1). Waktu untuk operasi swap dan pemanggilan adalah konstan c_1.

$$T(k) = k \cdot T(k - 1) + c_1$$

2. Base Case:

Ketika permutasi selesai dibentuk kondisi 1 == r yang ekuivalen dengan T(1), algoritma mengeksekusi loop untuk menghitung jarak rute. Loop ini berjalan melintasi array berukuran k dengan operasi penjumlahan jarak berbiaya c_2, lalu ditambah operasi evaluasi minimum berbiaya c_3.

$$T(1) = c_2 \cdot k + c_3 = O(k)$$

3. Ekspansi Metode Substitusi:

Tahap ini perlu ekspansi persamaan rekurensinya hingga mencapai kondisi basis sebagai berikut.

$$T(k) = k \cdot T(k - 1) + c_1$$

$$T(k) = k \cdot [(k - 1) \cdot T(k - 2) + c_1] + c_1$$

$$T(k) = k(k - 1) \cdot T(k - 2) + k \cdot c_1 + c_1$$

Jika diekspansi terus-menerus sebanyak k - 1 kali hingga menyentuh T(1) maka bentuk akhirnya adalah sebagai berikut.

$$T(k) = k! \cdot T(1) + c_1 \sum_{i=1}^{k-1} \frac{k!}{i!}$$

Kemudian substitusi nilai T(1) = O(k) ke dalam persamaan di atas sebagai berikut.

$$T(k) = k! \cdot O(k) + O(k!)$$

$$T(k) = O(k \cdot k!)$$

4. Penentuan Batas Asimptotik Big-O:

Substitusi kembali nilai asli k = n - 1 sebagai berikut.

$$T(n) = O((n - 1) \cdot (n - 1)!)$$

Berdasarkan sifat faktorial yang mana n · (n - 1)! = n!, maka dapat disederhanakan ekspresi matematikanya sebagai berikut.

$$T(n) = O(n!)$$

Kesimpulannya adalah algoritma ini memiliki kompleksitas batas atas asimptotik O(n!) yang merepresentasikan kompleksitas faktorial yang secara matematis membuktikan bahwa pendekatan Brute Force sangat tidak efisien untuk n berskala besar.

b. Analisis Kompleksitas Ruang (Space Complexity Analysis)

Kompleksitas ruang dihitung dari total memori yang digunakan ($S_{total} = S_{input} + S_{auxiliary}$).

1. Ruang Input (S_{input}):

Matriks representasi jarak D berdimensi n x n.

$$S_{input}(n) = O(n^2)$$

2. Ruang Tambahan / Auxiliary Space (S_{aux}):

Ruang tambahan dibentuk dari struktur data pembantu dan kedalaman Call Stack memori pada fungsi rekursif. Alokasi array kotaTersisa dan ruteTerbaik masing-masing dideklarasikan satu kali dengan ukuran n - 1.

$$S_{array} = 2 \times (n - 1) \times O(1) = O(n)$$

Ruang Stack Rekursi ini fungsi rekursi memanggil dirinya sendiri hingga menyentuh base case. Rentang indeks 1 bergerak dari 0 hingga n - 2, sehingga maksimum kedalaman pohon rekursi (maximum depth of recursion tree) adalah n - 1. Pada setiap kedalaman (frame stack), algoritma hanya menyimpan variabel primitif lokal yang konstan O(1).

$$S_{stack} = (n - 1) \times O(1) = O(n)$$

Maka persamaan Auxiliary Space adalah sebagai berikut.

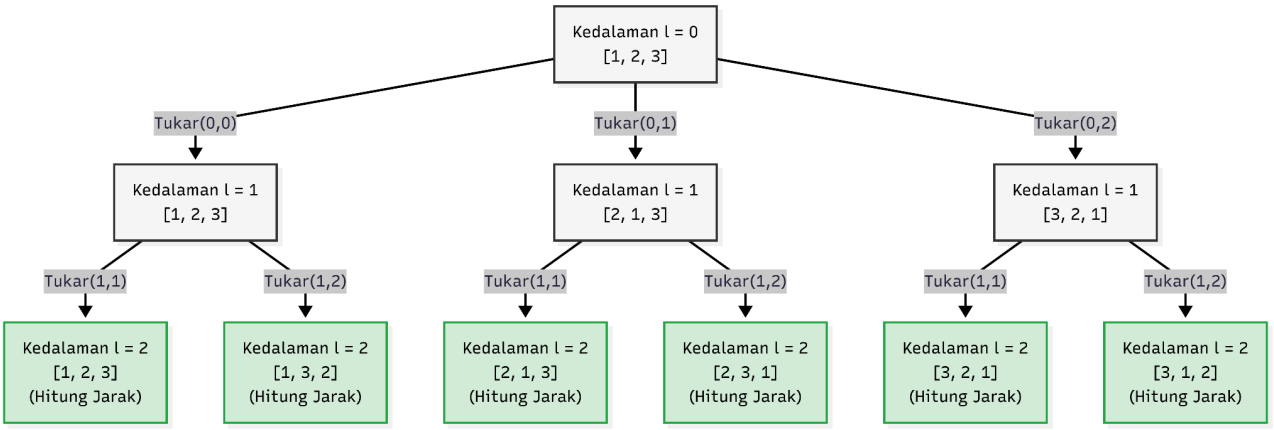
$$S_{aux}(n) = S_{array} + S_{stack} = O(n) + O(n)$$

$$S_{aux}(n) = O(n)$$

Kesimpulannya adalah karena algoritma mengadopsi teknik in place swapping dengan menukar kembali elemen ke tempat asalnya alih-alih mengalokasikan memori array baru untuk setiap permutasi dengan kompleksitas ruang tambahannya (auxiliary space) berbanding lurus secara linear terhadap jumlah kota yaitu O(n). Tentunya ini merupakan pencapaian penggunaan memori yang sangat optimal.

4. Analisis Diagram Pohon Rekursi

Disini saya juga mengilustrasikan eksekusi fungsi cariPermutasi untuk 3 kota tersisa misalnya kota 1, 2, dan 3 setelah kota 0 dikunci sebagai titik awal. Akar Pohon terdapat pada Kedalaman l =0 sebagai state awal saat fungsi pertama kali dipanggil. Array masih dalam urutan awal [1, 2, 3], pada titik ini program bersiap menukar elemen indeks ke 0 dengan indeks ke 0, 1, dan 2. Berikutnya Cabang Operasi bernama Tukar ini garis panah abu-abu merepresentasikan perulangan for dan operasi tukar(A, l, i) di dalam pseudocode. Misalnya pada jalur paling kiri Tukar(0,0) artinya elemen indeks 0 tetap di tempatnya, kemudian level berikutnya Tukar(1,1) berarti elemen indeks 1 juga tetap. Selanjutnya Simpul Daun atau Base Case yang ditandai kotak hijau di kedalaman l = 2 ini adalah kondisi ketika 1 == r terpenuhi. Pada titik ini algoritma berhenti melakukan rekursi (berhenti bercabang) karena satu rute permutasi yang utuh telah terbentuk. Pada kotak hijau inilah algoritma melakukan perulangan untuk mengeksekusi perhitungan jarak total rute yang sudah terlampir pada Gambar 2.



Gambar 2. Ilustrasi Kedalaman Memori (Call Stack) pada Pohon Rekursi

Pembuktian kompleksitas waktu yang hasilnya tadi adalah O(n!) ini terbukti pada jumlah 6 kotak hijau yang merupakan hasil dari 3! hasilnya 6. Apabila memiliki 4 kota untuk dipermutasi, jumlah daunnya akan melebar menjadi 24 kotak yang asalnya dari 4!. Hal ini yang dimaksud dengan perhitungan waktu O(n!), artinya seiring bertambahnya kedalaman pohon maka jumlah cabang yang harus dievaluasi akan bertambah secara faktorial yang membuktikan bahwa pendekatan Brute Force sangat memakan waktu.

Selain kompleksitas waktu, pembuktian kompleksitas ruang O(n) ini meskipun ujung bawah pohon sangat lebar ada 6 kemungkinan, memori komputer tidak menyimpan bentuk pohon ini secara utuh pada satu waktu. Eksekusi program berjalan secara Depth-First yang mana komputer akan mengeksekusi jalur paling kiri dahulu yaitu Root -> Tukar(0,0) -> Tukar(1,1) -> Kotak Hijau [1,2,3]. Pada titik ini memori Call Stack hanya menumpuk 3 pemanggilan fungsi karena tingginya hanya 3 level. Setelah jarak [1,2,3] dihitung, komputer menghapus stack tersebut, kemudian naik ke atas, dan turun ke cabang Tukar(1,2) untuk mengevaluasi [1,3,2]. Karena komputer hanya menahan satu garis lurus memori dari akar ke daun pada satu waktu, maka penggunaan ruang tambahannya (auxiliary space) berbanding lurus dengan tinggi pohon kedalaman maksimum yaitu linear O(n).